Laboratory Exercise 2: Algorithms, Control Flow & Modular Functions

Student Details	
Student Name: _____	**Student ID:** _____
Duration: 2 Hours (120 Minutes)	**Total Points:** 100 Points

1. Learning Objectives

- **Control Flow & Debugging:** Translate pseudocode into Python selection statements (`if – elif – else`) and resolve conditional logic errors.
- **Functions & Tuples:** Design modular functions with default parameters, return multiple values via tuples, and unpack them cleanly.
- **Stochastic Modeling:** Utilize the `random` module to construct probabilistic engineering simulations (Monte Carlo wind shear model).

2. Exercise Structure, Evaluation & Instructor Sign-Off

This 120-minute laboratory session is divided into 5 sequential parts totaling 100 points: **Part 1: Pseudocode Translation & Boolean Logic** (20 mins, 15 pts), **Part 2: Multi-way Branching & Logic Error Debugging** (25 mins, 20 pts), **Part 3: Function Architecture & Multiple Return Values** (25 mins, 20 pts), **Part 4: Practical Modeling: Runway Assignment & Crosswind Limits** (30 mins, 25 pts), and **Part 5: Challenge: Stochastic Flight Approach Simulation** (20 mins, 20 pts).

Instructor Verification	Details
Score Awarded: _____ / 100 Points	**Instructor Signature:** _____ **Date:** _____ / _____ / 20__
Instructor Comments & Feedback:	_____

Part 1: Pseudocode Translation & Boolean Logic (15 Points)

Task 1.1: Pseudocode to Python Implementation (8 Points)

In Lecture 3, you learned that an algorithm is an unambiguous sequence of steps. Consider the following pseudocode designed to determine whether an aircraft is cleared for Visual Flight Rules (VFR) based on flight visibility (in statute miles) and ceiling height (in feet):

```text
Algorithm VFR_Clearance:
1. Input visibility, ceiling
2. IF visibility >= 3.0 AND ceiling >= 1000 THEN
3.     status = "VFR Cleared"
4. ELSE
5.     status = "IFR Required (Instrument Flight Rules)"
6. Output status
```

1. **(4 Points)** Write a Python code snippet that implements this exact logic. Test with `visibility = 5.0` and `ceiling = 1200`.

   Answer / Status Output: _____

2. **(4 Points)** Re-write the condition into a single-line **conditional expression** (ternary operator) in Python:

   ```python
   status = "VFR Cleared" if ... else "IFR Required"
   ```

   Single-line Expression: _____

Task 1.2: Boolean Short-Circuit Evaluation (7 Points)

Evaluate the truth value of the following boolean expressions without running them first, then verify with Python:

```python
a = 15
b = 0
c = 25

exp1 = (a > 10) and (c < 30)
exp2 = (b != 0) and (c / b > 1)  # Does this cause a ZeroDivisionError? Why or why not?
exp3 = (a == 15) or (b != 0 and c == 25)
```

Student Answer / Evaluation:

- `exp1` : _____ | `exp2` : _____ | `exp3` : _____
- **Explanation of Short-Circuit Evaluation (`exp2`):**

  _____
  _____

Part 2: Multi-way Branching & Logic Error Debugging (20 Points)

Task 2.1: Wake Turbulence Separation Classifier (10 Points)

Air Traffic Control classifies aircraft into Wake Turbulence Categories based on Maximum Takeoff Weight (MTOW in kilograms):

- **Super (J):** MTOW $\geq 500,000$ kg (e.g., Airbus A380)
- **Heavy (H):** $136,000 \text{ kg} \leq \text{MTOW} < 500,000 \text{ kg}$ (e.g., Boeing 777)
- **Medium (M):** $7,000 \text{ kg} < \text{MTOW} < 136,000 \text{ kg}$ (e.g., Airbus A320)
- **Light (L):** $\text{MTOW} \leq 7,000$ kg (e.g., Cessna 172)

Write an interactive script that prompts the user for the aircraft MTOW and outputs its category code and description.

Example Run:

```text
Enter aircraft MTOW in kg: 78000
Wake Category: [M] Medium (7,000 kg < MTOW < 136,000 kg)
```

Student Answer / Program Output:

- **Aircraft MTOW Tested:** _____ kg
- **Wake Category Code & Description:** _____

Task 2.2: Debugging Logic Errors (10 Points)

A student submitted the following code intended to assign fuel reserve ratings based on remaining flight time in minutes. It runs without syntax errors, but contains a serious **logic error** in its ordering:

```python
# BUGGY CODE:
fuel_minutes = 45

if fuel_minutes >= 30:
    status = "Normal Reserve"
elif fuel_minutes >= 45:
    status = "Optimal Reserve"
elif fuel_minutes < 30:
    status = "Low Fuel Warning"
```

```
elif fuel_minutes < 15:
    status = "EMERGENCY: Minimum Fuel"
```

1. **(4 Points)** Identify why the status for `fuel_minutes = 10` or `fuel_minutes = 50` produces the wrong output.

   Identification of Logic Error:

   _____
   _____

2. **(6 Points)** Rewrite the `if – elif – else` block so that all conditions execute in the correct logical hierarchy.

   Expected Outputs with Correct Logic:

   - `fuel_minutes = 50` : _____ | `fuel_minutes = 35` : _____
   - `fuel_minutes = 20` : _____ | `fuel_minutes = 10` : _____

   Corrected Hierarchy / Ordering Explanation: _____

Part 3: Function Architecture & Multiple Return Values (20 Points)

Task 3.1: True Airspeed & Mach Number Function (10 Points)

Write a Python function named `compute_flight_speeds(ias, altitude_ft, temp_c)` :

- **Inputs:** `ias` (Indicated Airspeed, kt), `altitude_ft` (Pressure alt, ft), `temp_c` (OAT, °C)
- **Formulas:**

$$\text{TAS} = \text{ias} \times \left(1 + \frac{\text{altitude_ft}}{1000} \times 0.02\right), \qquad a = 38.967 \times \sqrt{\text{temp_c} + 273.15}, \qquad M$$

- **Output:** Return both `tas` and `mach` as a **tuple**.

Task 3.2: Calling and Unpacking (10 Points)

Call `compute_flight_speeds(ias=250.0, altitude_ft=32000.0, temp_c=-45.0)` . Unpack the returned tuple into two variables, `true_airspeed` and `mach_number` , and display the results formatted to 2 decimal places.

Student Answer / Program Output:

- **Calculated TAS:** _____ knots | **Speed of Sound (a):** _____ knots
- **Calculated Mach Number:** _____ | **Returned Tuple (`tas` , `mach`):** `(_____, _____)`

Part 4: Practical Modeling: Runway Assignment & Crosswind Limits (25 Points)

When landing, an aircraft experiences a **Crosswind Component** (V_{cross}) and a **Headwind Component** (V_{head}):

$$V_{\text{cross}} = V_{\text{wind}} \times |\sin(\theta_{\text{diff}})|, \qquad V_{\text{head}} = V_{\text{wind}} \times \cos(\theta_{\text{diff}})$$

Where:

- V_{wind} = Wind speed (knots)
- $\theta_{\text{diff}} = \text{Wind Direction} - \text{Runway Heading}$ (in radians: `math.radians(degrees)`)

Requirements:

Write a function `check_landing_clearance(runway_heading, wind_dir, wind_speed, max_crosswind=15.0)` that:

1. Calculates V_{cross} and V_{head}.
2. Evaluates clearance:
   - If $V_{\text{head}} < 0$, it is a **tailwind**; if tailwind exceeds 10.0 knots, landing is **"REJECTED: Excessive Tailwind"**.
   - If $V_{\text{cross}} > \text{max_crosswind}$, landing is **"REJECTED: Crosswind Limit Exceeded"**.
   - Otherwise, landing is **"CLEARED: Safe Landing Conditions"**.
3. Returns a tuple `(clearance_status, crosswind, headwind)`.

Prompt the user to enter parameters interactively and print the flight operations dispatch report.

Example Run:

```
text
```

```
Enter runway heading (degrees): 090
Enter wind direction (degrees): 130
Enter wind speed (knots): 22

--- Dispatch Report ---
Headwind Component: _____ knots
Crosswind Component: ______ knots
Clearance: CLEARED: ______
```

Student Answer / Program Output:

- **Runway Heading:** _____ deg | **Wind Direction:** _____ deg | **Wind Speed:** _____ kt
- **Headwind Component:** _____ knots | **Crosswind Component:** _____ knots
- **Clearance Status:** _____

Part 5: Challenge Task — Stochastic Wind Turbulence Simulation (20 Points)

In Lecture 4, you studied random number generation and dice roll simulation. In aviation engineering, **Monte Carlo simulations** are used to model turbulent wind gusts.

Write a simulation program that:

1. Prompts the user for the number of approach trials (e.g., $N = 1000$).
2. Uses a `for` loop and `random.randint()` or `random.uniform()` to simulate wind gusts during each approach:
   - Base steady wind: 15.0 knots
   - Gust variation: Random integer between -5 and $+15$ knots added to base wind.
3. If total wind (base + gust) strictly exceeds 25.0 knots, the approach experiences a **Wind Shear Alarm**.
4. Count total alarms across all N trials.
5. Compute and print the empirical probability of a Wind Shear Alarm:

$$P(\text{Alarm}) = \frac{\text{Total Alarms}}{N} \times 100\%$$

Example Run:

```text
Enter number of approach trials: 5000

--- Monte Carlo Simulation Results ---
Total Approaches: _____
Wind Shear Alarms: _____
Empirical Alarm Probability: ______%
Safety Assessment: ______
```

(If $P(\text{Alarm}) > 20\%$, display `"HIGH CAUTION"`; otherwise display `"NORMAL OPERATIONS"`).

Student Answer / Program Output:

- **Total Approaches (N):** _____ | **Wind Shear Alarms:** _____
- **Empirical Alarm Probability ($P(\text{Alarm})$):** _____%
- **Safety Assessment Message:** _____